



รายงาน

ระบบการขายหน้าร้านสำหรับร้านกาแฟ

Cafe Point of Sale System

จัดทำโดย

673380390-5 นางสาวกวิณริตา อนุนิวัฒน์

673380418-9 นายภควัฒน์ สุขมณี

673380573-7 นางสาวกัญญาวีร์ สิงห์ลี

673380586-8 นายธนนันต์ สาวิกัน

เสนอ

ผศ.ดร.ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของรายวิชา

CP353002 Principles of Software Design and Development

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

เรื่อง: ระบบการขายหน้าร้านสำหรับร้านกาแฟ (Café Point of Sale System)

รายวิชา: CP353002 Principles of Software Design and Development

คณะผู้จัดทำ:	1. นางสาวกวิณธิดา	อนุนิวัฒน์
	2. นายภควัฒน์	สุขมณี
	3. นางสาวกัญญาวิรี	สิงห์ลี
	4. นายธนันต์	สาวิกัน

อาจารย์ประจำวิชา: รศ.ดร.ปัญญาพล หอระตะ

สถานศึกษา: มหาวิทยาลัยขอนแก่น

ปีการศึกษา: 2569

บทคัดย่อ

โครงการนี้มีวัตถุประสงค์เพื่อออกแบบและพัฒนาระบบจุดขายสำหรับร้านกาแฟ (Cafe POS) ด้วยสถาปัตยกรรมแบบ Layered Architecture โดยใช้ Spring Boot 4.1 (Java 21) ฝั่ง Backend และ React 18 ร่วมกับ Vite ฝั่ง Frontend เก็บข้อมูลใน PostgreSQL 17 ที่จัดการโครงสร้างด้วย Flyway และยืนยันตัวตนด้วย JWT แบบ Stateless ระบบแบ่งผู้ใช้เป็น 2 บทบาท คือ Cashier และ Admin ซึ่ง Admin สืบทอดสิทธิ์ทั้งหมดของ Cashier ฟังก์ชันหลักครอบคลุมการเลือกสินค้าตามหมวดหมู่ การเลือกตัวเลือกเสริม (Add-ons) การสร้างและแก้ไขออเดอร์ การใช้ส่วนลดแบบกำหนดเองหรือจากโปรโมชั่น การรับชำระเงินด้วย เงินสดและ QR Code การออกและพิมพ์ใบเสร็จ การค้นหาบิลย้อนหลัง การจัดการข้อมูลหลัก (สินค้า หมวดหมู่ Add-on โพรโมชัน ผู้ใช้) และรายงานสรุปยอดขายสำหรับ Admin

ด้านการออกแบบ โครงการประยุกต์ใช้หลัก SOLID และ Design Patterns ได้แก่ Strategy สำหรับการคำนวณส่วนลด State สำหรับควบคุมสถานะออเดอร์ (PENDING, PAID, CANCELLED) และ Observer ผ่าน Spring Application Event เมื่อชำระเงินสำเร็จ รวมทั้งใช้แนวคิด Snapshot เก็บราคาสินค้า ราคา Add-on และเงื่อนไขส่วนลด ณ เวลาขาย เพื่อให้ประวัติการขายไม่เปลี่ยนแปลง เมื่อมีการแก้ไขราคากายหลัง ระบบมี Automated Test 21 คลาส (ประมาณ 119 กรณีทดสอบ) ด้วย JUnit 5 และ Mockito มีกระบวนการ CI ด้วย GitHub Actions และบรรจุเป็น Docker image เดียวเพื่อ Deploy บน Render ร่วมกับฐานข้อมูล Neon

คำสำคัญ: ระบบจุดขาย, Spring Boot, Layered Architecture, SOLID, Design Pattern, JWT, React

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อประกอบรายวิชา CP353002 Principles of Software Design and Development โดยมีเป้าหมายเพื่อให้ผู้เรียนสามารถนำความรู้ด้านการวิเคราะห์ ออกแบบและพัฒนาซอฟต์แวร์มาประยุกต์กับระบบที่ใช้งานได้จริง คณะผู้จัดทำเลือกพัฒนาระบบ Cafe POS เพื่อลดความผิดพลาดและความล่าช้าในการรับออเดอร์และคิดเงินของร้านกาแฟ

เนื้อหาประกอบด้วย การวิเคราะห์และออกแบบด้วยแผนภาพ UML (Use Case, Domain Model, Class, Activity, Sequence, State, Component, Deployment) แผนภาพ ER และ Data Dictionary การประยุกต์ SOLID และ Design Patterns การออกแบบ REST API และการจัดการข้อผิดพลาด ตลอดจนกระบวนการ Git Workflow การทดสอบ และการ Deploy

คณะผู้จัดทำหวังว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ศึกษา หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขอน้อมรับคำแนะนำเพื่อนำไปปรับปรุงต่อไป

คณะผู้จัดทำ

ตุลาคม 2569

กิตติกรรมประกาศ

รายงานโปรเจกต์ระบบการขายหน้าร้านสำหรับร้านกาแฟ (Cafe POS) ฉบับนี้สำเร็จลุล่วงได้ด้วยความรู้ความกรุณาและคำแนะนำจาก ผศ.ดร.ปัญญาพล หอระตะ อาจารย์ประจำรายวิชา CP353002 Principles of Software Design and Development ที่ได้ให้ความรู้ แนวทางการออกแบบ และข้อเสนอแนะในการพัฒนาระบบด้วย Spring Boot ตามหลักวิศวกรรมซอฟต์แวร์

คณะผู้จัดทำขอขอบคุณเพื่อน ๆ ในรายวิชาที่ช่วยทดสอบระบบและแลกเปลี่ยนความคิดเห็นและขอบคุณสมาชิกในทีมทุกคนที่ร่วมกันวางแผน ออกแบบ พัฒนา ตรวจสอบโค้ด และแก้ไขปัญหาตลอดระยะเวลาของโปรเจกต์

คณะผู้จัดทำหวังว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจศึกษาการออกแบบระบบขายหน้าร้านรวมถึงการพัฒนาแอปพลิเคชันด้วย Spring Boot

คณะผู้จัดทำ

ตุลาคม 2569

สารบัญ

เรื่อง	หน้า
บทคัดย่อ	ก
คำนำ	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
บทที่ 1 บทนำ	
1.1 ความเป็นมาและความสำคัญ	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตของระบบ	2
บทที่ 2 ภาพรวมระบบและเทคโนโลยีที่ใช้	
2.1 เทคโนโลยีที่เลือกใช้	3
2.2 สถาปัตยกรรมแบบ Layered Architecture	3
2.3 ความปลอดภัยและการควบคุมสิทธิ์	4
2.4 Component Diagram	5
บทที่ 3 การวิเคราะห์และออกแบบระบบ	
3.1 Use Case Diagram และ Use Case Description	6

3.2 Domain Model	23
3.3 Class Diagram และ Design Patterns	23
3.4 Activity Diagram	25
3.5 Entity Relationship Diagram และ Data Dictionary	26
3.6 Sequence Diagram	33
3.7 State Diagram	36
3.8 Deployment Diagram	37
บทที่ 4 การประยุกต์ใช้หลักการออกแบบและ Design Patterns	
4.1 รายงานการวิเคราะห์ SOLID Principles	38
4.2 รายงานการประยุกต์ใช้ Design Patterns	39
บทที่ 5 REST API และการจัดการข้อผิดพลาด	
5.1 รายละเอียด RESTful API Endpoints	42
5.2 การตรวจสอบข้อมูลและการจัดการ Exception	44
5.3 Database Migration	46
บทที่ 6 กระบวนการทำงาน การทดสอบ และการ Deploy	
6.1 Git Workflow และ Branching Strategy	47

6.2 การทดสอบระบบ	47
6.3 Docker, CI และการ Deploy	49
เอกสารอ้างอิง	๕

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญ

ธุรกิจร้านกาแฟและคาเฟ่ในปัจจุบันต้องรองรับปริมาณธุรกรรมการขายจำนวนมากในช่วงเวลาที่เร่งด่วน การดำเนินงานด้วยระบบมือ (Manual Operations) เช่น การจดบันทึกรายการออเดอร์และการคำนวณเงินด้วยตนเอง มักก่อให้เกิดข้อผิดพลาดได้ง่าย เช่น การคิดราคาตัวเลือกเสริม (Add-ons) หรือการประยุกต์ใช้ส่วนลดผิดพลาด รายการในใบเสร็จรับเงินไม่ตรงกับสินค้าที่จัดจำหน่ายจริง ตลอดจนความยากลำบากในการตรวจสอบและสรุปยอดขายย้อนหลัง ระบบบริหารจัดการจุดขาย (Point of Sale: POS) จึงเป็นเครื่องมือสำคัญที่เข้ามาช่วยเพิ่มประสิทธิภาพในการรับออเดอร์ การชำระเงิน การออกใบเสร็จ และการสรุปยอดขายให้เป็นไปอย่างรวดเร็ว แม่นยำ และสามารถตรวจสอบย้อนหลังได้

ด้วยเหตุนี้ โครงการงาน Cafe POS System จึงถูกพัฒนาขึ้นในรูปแบบเว็บแอปพลิเคชันที่มีการสถาปัตยกรรมแบบแยกส่วนระหว่างฝั่งหน้าบ้าน (Frontend) และหลังบ้าน (Backend) โดยสื่อสารข้อมูลกันผ่าน RESTful API การออกแบบซอฟต์แวร์เน้นย้ำการแบ่งชั้นความรับผิดชอบอย่างชัดเจน (Layered Architecture) ร่วมกับการประยุกต์ใช้หลักการออกแบบ SOLID Principles และ GoF Design Patterns เพื่อให้โครงสร้างระบบมีความยืดหยุ่น รองรับการขยายตัวในอนาคต และง่ายต่อการบำรุงรักษาในระยะยาว

1.2 วัตถุประสงค์

- 1.2.1 ออกแบบและพัฒนาระบบ POS สำหรับร้านกาแฟด้วย Spring Boot (Backend) และ React (Frontend) ตามสถาปัตยกรรม Layered Architecture
- 1.2.2 ประยุกต์ใช้หลักการ SOLID และ Design Patterns (Strategy, State, Observer) ในส่วนที่เหมาะสมกับปัญหาจริงของระบบ
- 1.2.3 ออกแบบฐานข้อมูลเชิงสัมพันธ์ที่มีความสัมพันธ์ครบทั้ง One-to-One, One-to-Many และ Many-to-Many และใช้ Flyway จัดการ Migration
- 1.2.4 เก็บข้อมูลราคาและส่วนลดแบบ Snapshot เพื่อรักษาความถูกต้องของประวัติการขาย
- 1.2.5 จัดทำ Automated Test, CI และ Deploy ระบบด้วย Docker บนระบบคลาวด์

1.3 ขอบเขตของระบบ

1.3.1 ขอบเขตด้านฟังก์ชันการทำงาน (Functional Scope)

1. ส่วนของผู้ใช้งาน/ลูกค้า

- การสมัครสมาชิก เข้าสู่ระบบ และจัดการข้อมูลส่วนตัว
- การค้นหา เรียกดูรายการ และดูรายละเอียดของสินค้าหรือบริการ
- การทำรายการสั่งซื้อ ชำระเงิน และแจ้งชำระเงิน
- การติดตามสถานะการสั่งซื้อหรือการจัดส่ง
- การให้คะแนน รีวิว หรือส่งข้อความสอบถาม

2. ส่วนของผู้ดูแลระบบ (Admin)

- การจัดการข้อมูลผู้ใช้งาน (เพิ่ม ลบ แก้ไข หรือระงับสิทธิ์)
- การจัดการข้อมูลสินค้า/บริการ และคลังสินค้า
- การตรวจสอบและอนุมัติรายการสั่งซื้อและการชำระเงิน
- การจัดการข้อมูลการจัดส่งและอัปเดตสถานะ
- การออกรายงานสรุยอดขาย ประวัติการใช้งาน หรือสถิติต่างๆ

1.3.2 ขอบเขตและข้อจำกัดของระบบ (System Constraints & Non-Functional Scope)

1. ข้อจำกัดด้านอุปกรณ์และแพลตฟอร์ม: ระบบรองรับการใช้งานผ่าน Web Browser (เช่น Chrome, Safari) บนคอมพิวเตอร์และอุปกรณ์มือถือ
2. ข้อจำกัดด้านผู้ใช้งาน: ระบบถูกออกแบบมาเพื่อรองรับผู้ใช้งานพร้อมกันได้ไม่เกินจำนวนที่กำหนด (เช่น รองรับผู้ใช้พร้อมกันสูงสุด 1,000 คน)
3. ข้อจำกัดด้านระบบชำระเงิน: รองรับชำระเงินผ่านวิธีที่กำหนดเท่านั้น เช่น โอนเงินผ่านธนาคาร (QR Code) และบัตรเครดิต/เดบิต
4. ข้อจำกัดด้านภาษา: ระบบรองรับการแสดงผลภาษาไทยเป็นหลัก (และภาษาอังกฤษในบางส่วน)
5. สิ่งที่ระบบไม่รองรับ (Out of Scope): ระบบไม่รวมถึงขั้นตอนการจัดส่งสินค้าจริงโดยบริษัทขนส่งภายนอก (ระบบจัดการเฉพาะการบันทึกหมายเลขพัสดุเท่านั้น)

บทที่ 2

ภาพรวมระบบและเทคโนโลยีที่ใช้

2.1 เทคโนโลยีที่เลือกใช้

ตารางที่ 1 เทคโนโลยีที่ใช้ในโปรเจก

ส่วน	เทคโนโลยี	หมายเหตุ
Backend	Spring Boot 4.1.1, Java 21, Maven	REST API, Bean Validation, Actuator
Security	Spring Security (OAuth2 Resource Server), JWT HS256, BCrypt	Stateless, ตรวจสอบสิทธิ์จาก claim roles
Database	PostgreSQL 17, Flyway	Migration V1–V5, H2 สำหรับ Test
ORM	Spring Data JPA (Hibernate), Specification	Pagination, Sorting และ Dynamic Filter
API Docs	springdoc-openapi 3.1.1 (Swagger UI)	/swagger-ui.html, /v3/api-docs
Frontend	React 18.3, Vite 5, React Router 6	Single Page Application
Testing	JUnit 5, Mockito, Spring Boot Test	Unit Test และ Integration Test
DevOps	Docker (multi-stage), Docker Compose, GitHub Actions, Render, Neon	Image เดียว ไม่ต้องตั้งค่า CORS

2.2 สถาปัตยกรรมแบบ Layered Architecture

Backend แบ่งเป็นชั้นตามหน้าที่ โดยให้การเรียกไหลในทิศทางเดียว คือ Controller → Service → Repository → Entity/Database ไม่มีการเรียกข้ามชั้น Controller ไม่เรียก Repository ตรง และ Entity ไม่ถูกส่งออกไปยัง API โดยตรงแต่ผ่าน DTO และ Mapper

```

com.cafepos
├── config/      SecurityConfig (JWT), SpaConfig
├── controller/api/ REST Controllers (9 ตัว)
├── service/     Service interfaces
│   ├── impl/   Business logic, @Transactional
│   ├── discount/ DiscountStrategy + 3 impl
│   └── listener/ SaleAuditLogListener (Observer)
├── repository/  Spring Data JPA + Specification
├── domain/
│   ├── entity/  JPA entities (10 คลาส)
│   ├── enums/   Role, OrderStatus, PaymentMethod,
│   │           DiscountType
│   ├── state/   OrderState + 3 สถานะ
│   └── event/   OrderPaidEvent
├── dto/         request/, response/
├── mapper/      Entity <-> DTO
├── exception/   GlobalExceptionHandler
└── common/     CurrentUser, ShopTime

```

2.3 ความปลอดภัยและการควบคุมสิทธิ์

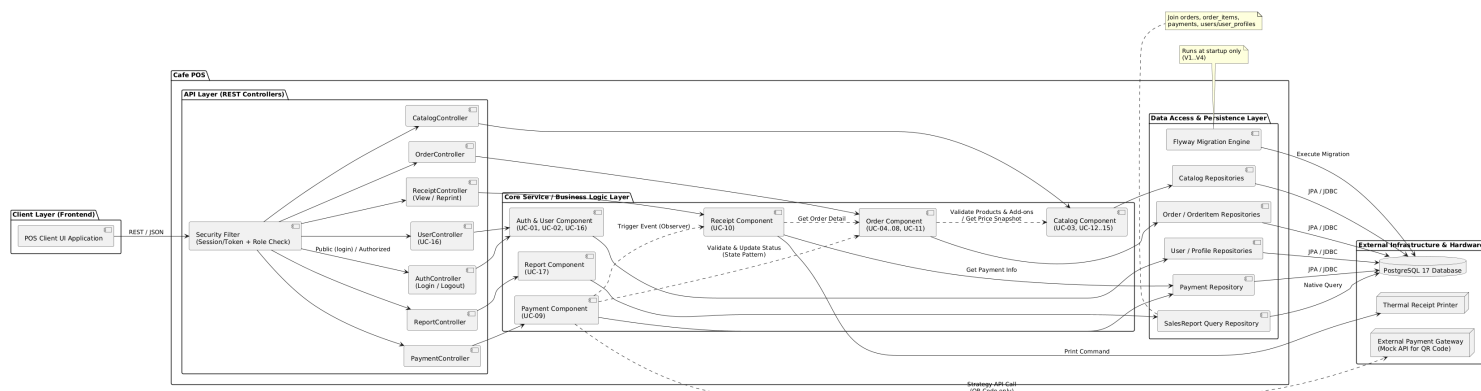
ผู้ใช้เข้าสู่ระบบที่ POST /api/v1/auth/login ระบบตรวจรหัสผ่านที่เก็บแบบ BCrypt แล้วออก JWT (HS256) ที่มี claim roles ทุก Request อื่นได้ /api ต้องแนบ Authorization: Bearer <token> ระบบไม่ใช่ Session และ Cookie จึงปิด CSRF ได้ การกำหนดสิทธิ์อยู่ใน SecurityConfig ดังตารางที่ 2.2

ตารางที่ 2.2 สิทธิ์การเข้าถึง API ตามบทบาท

กลุ่ม API	Cashier	Admin
/auth/login	ทุกคน (ไม่ต้องล็อกอิน)	ทุกคน
/users/me/** (โปรไฟล์ เปลี่ยนรหัสผ่าน)	ได้	ได้
/orders/**, /orders/{id}/payment	ได้ (เห็นเฉพาะบิลของตนเอง)	ได้ (เห็นทุกบิล)
GET/products, /categories, /add-ons, /promotions	ได้	ได้
POST/PUT/PATCH/DELETE ข้อมูลหลักข้างต้น	ไม่ได้ (403)	ได้
/users/** (จัดการผู้ใช้), /reports/**	ไม่ได้ (403)	ได้

2.4 Component Diagram

Frontend เป็น React SPA ที่เรียก REST API ผ่านโมดูล api/ และเก็บสถานะการล็อกอินใน AuthProvider ส่วน Backend ผ่าน Security Filter ก่อนถึง Controller แล้วไหลตามชั้นไปยังฐานข้อมูล PostgreSQL



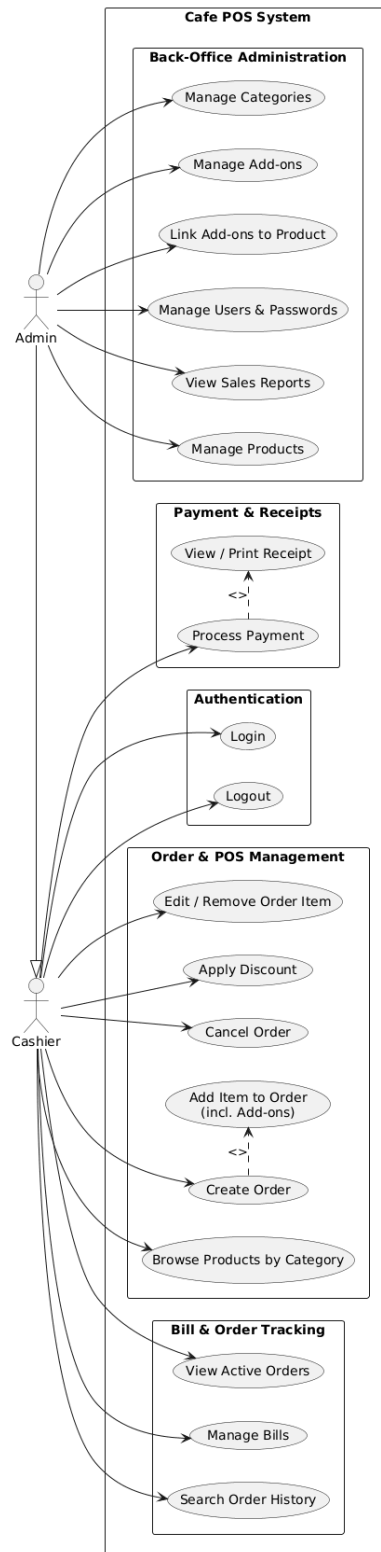
ภาพที่ 2.1 Component Diagram

บทที่ 3

การวิเคราะห์และออกแบบระบบ

3.1 Use Case Diagram และ Use Case Description

ระบบมี Actor 2 ตัว คือ Cashier และ Admin โดย Admin สืบทอดสิทธิ์ของ Cashier ทั้งหมด (Generalization) แผนภาพแบ่งเป็น 2 ภาพเพื่อให้อ่านง่าย ภาพที่ 3.1 แสดงงานของ Cashier และภาพที่ 3.2 แสดงงานหลังบ้านของ Admin มีความสัมพันธ์ «include» 2 จุด คือ Create Order รวม Add Item to Order และ Process Payment รวม View / Print Receipt



ภาพที่ 3.1 Use Case Diagram

ตารางที่ 3.1 สรุป Use Case ทั้งหมด

รหัส	Use Case	กลุ่ม	Actor
UC-01	Login	Authentication & Profile	Cashier, Admin
UC-02	Logout	Authentication & Profile	Cashier, Admin
UC-03	View / Edit Profile	Authentication & Profile	Cashier, Admin
UC-04	Change Password	Authentication & Profile	Cashier, Admin
UC-05	Create Order	Order & POS Management	Cashier (Admin สืบ ทอด)
UC-06	Browse Products by Category (incl. Search)	Order & POS Management	Cashier (Admin สืบ ทอด)
UC-07	Add Item to Order	Order & POS Management	Cashier (Admin สืบ ทอด)
UC-08	Select Add-ons & Sweetness Level	Order & POS Management	Cashier (Admin สืบ ทอด)
UC-09	Apply Promotion	Order & POS Management	Cashier (Admin สืบ ทอด)
UC-10	Edit / Remove Order Item	Order & POS Management	Cashier (Admin สืบ ทอด)
UC-11	Cancel Order	Order & POS Management	Cashier (Admin สืบ ทอด)
UC-12	Process Payment	Payment & Receipts	Cashier (Admin สืบ ทอด)
UC-13	Select Payment Method (Cash / QR)	Payment & Receipts	Cashier (Admin สืบ ทอด)
UC-14	View / Print Receipt	Payment & Receipts	Cashier (Admin สืบ ทอด)

UC-15	Search Order History	Order History	Cashier (Admin สืบ ทอด)
UC-16	View Order Detail	Order History	Cashier (Admin สืบ ทอด)
UC-17	Reprint Receipt	Order History	Cashier (Admin สืบ ทอด)
UC-18	Manage Products	Back-Office Administration	Admin
UC-19	Manage Categories	Back-Office Administration	Admin
UC-20	Manage Add-ons	Back-Office Administration	Admin
UC-21	Link Add-ons to Product	Back-Office Administration	Admin
UC-22	Manage Promotions	Back-Office Administration	Admin
UC-23	View Sales Reports	Back-Office Administration	Admin
UC-24	Manage Users & Passwords	Back-Office Administration	

รายละเอียดของแต่ละ Use Case เป็นดังนี้

UC-01: Login

Actor	Cashier, Admin
Description	ผู้ใช้เข้าสู่ระบบด้วยชื่อผู้ใช้และรหัสผ่านเพื่อเริ่มใช้งาน
Pre-conditions	มีบัญชีผู้ใช้ในระบบและบัญชียังเปิดใช้งานอยู่
Post-conditions	ระบบสร้างเซสชันและแสดงหน้าจอตามสิทธิ์ (Cashier เห็นหน้า POS, Admin เห็นเมนูหลังบ้านเพิ่ม)
Main Success Scenario	1. ผู้ใช้เปิดหน้าเข้าสู่ระบบ
Extensions	• 3a. ข้อมูลไม่ถูกต้อง ระบบแสดงข้อความผิดพลาดและให้กรอกใหม่ • 3b. บัญชีถูกปิดใช้งาน ระบบปฏิเสธการเข้าสู่ระบบ

UC-02: Logout

Actor	Cashier, Admin
Description	ผู้ใช้ออกจากระบบเมื่อเลิกใช้งานหรือเปลี่ยนกะ
Pre-conditions	ผู้ใช้เข้าสู่ระบบอยู่
Post-conditions	เซสชันสิ้นสุดและระบบกลับไปหน้า Login
Main Success Scenario	1. ผู้ใช้กดออกจากระบบ
Extensions	• 3a. ผู้ใช้ยกเลิก ระบบคงอยู่ที่หน้าเดิม

UC-03: View / Edit Profile

Actor	Cashier, Admin
Description	ผู้ดูแลและแก้ไขข้อมูลส่วนตัวของตนเอง
Pre-conditions	ผู้ใช้งานเข้าสู่ระบบอยู่
Post-conditions	ข้อมูลโปรไฟล์ถูกอัปเดตตามที่แก้ไข
Main Success Scenario	1. ผู้ใช้เปิดหน้าโปรไฟล์
Extensions	• 4a. ผู้ใช้ยกเลิก ระบบคงข้อมูลเดิม • 5a. ข้อมูลไม่ครบหรือรูปแบบไม่ถูกต้อง ระบบแจ้งเตือนและไม่บันทึก

UC-04: Change Password

Actor	Cashier, Admin
Description	ผู้ใช้งานเปลี่ยนรหัสผ่านของตนเอง
Pre-conditions	ผู้ใช้งานเข้าสู่ระบบอยู่
Post-conditions	รหัสผ่านใหม่ถูกบันทึก (เข้ารหัส) และใช้ได้ในการเข้าสู่ระบบครั้งถัดไป
Main Success Scenario	1. ผู้ใช้เลือกเปลี่ยนรหัสผ่าน
Extensions	• 3a. รหัสผ่านเดิมไม่ถูกต้อง ระบบแจ้งเตือนให้กรอกใหม่ • 3b. รหัสผ่านใหม่ไม่ตรงกับช่องยืนยัน หรือไม่ผ่านเงื่อนไข ระบบแจ้งเตือนและไม่บันทึก

UC-05: Create Order

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	เริ่มสร้างออเดอร์ใหม่และรวบรวมรายการที่ถูกคำสั่ง มีความสัมพันธ์ «include» กับ UC-06 และ UC-07 และมี UC-09 เป็น «extend»
Pre-conditions	ผู้ใช้เข้าสู่ระบบอยู่
Post-conditions	ได้ออเดอร์สถานะรอชำระเงินที่มีรายการอย่างน้อย 1 รายการ
Main Success Scenario	1. ผู้ใช้เริ่มออเดอร์ใหม่
Extensions	• 6a. ออเดอร์ไม่มีรายการ ระบบไม่อนุญาตให้ไปขั้นตอนชำระเงิน

UC-06: Browse Products by Category (incl. Search)

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	แสดงสินค้าตามหมวดหมู่ (เช่น กาแฟ ชา ขนม) และค้นหาสินค้าจากชื่อ เป็นส่วนหนึ่งของ UC-05 («include»)
Pre-conditions	มีสินค้าที่เปิดใช้งานอยู่ในระบบ
Post-conditions	ผู้ใช้เห็นสินค้าที่ต้องการและเลือกสินค้าได้
Main Success Scenario	1. ระบบแสดงรายการหมวดหมู่ 2. ผู้ใช้เลือกหมวดหมู่ 3. ระบบแสดงสินค้าในหมวดนั้น 4. (ถ้าต้องการ) ผู้ใช้พิมพ์คำค้นหา ระบบกรองผลตามชื่อสินค้า
Extensions	• 4a. ไม่พบสินค้าตามคำค้นหา ระบบแจ้งว่าไม่พบสินค้า
หมายเหตุ	สินค้าที่ปิดใช้งานจะไม่แสดงในรายการ

UC-07: Add Item to Order

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	เพิ่มสินค้าที่เลือกลงในออเดอร์ พร้อมจำนวนและรายละเอียด เป็นส่วนหนึ่งของ UC-05 («include») และมี UC-08 เป็น «extend»
Pre-conditions	ผู้ใช้เลือกสินค้าแล้ว และมีออเดอร์ที่กำลังสร้างอยู่
Post-conditions	รายการสินค้าถูกเพิ่มลงออเดอร์ และยอดรวมถูกคำนวณใหม่
Main Success Scenario	1. ผู้ใช้แตะสินค้า
Extensions	• 5a. ผู้ใช้ปิดหน้าต่าง ระบบไม่เพิ่มรายการ
หมายเหตุ	• จำนวนต่ำสุดคือ 1

UC-08: Select Add-ons & Sweetness Level

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	เลือกตัวเลือกเสริมของเครื่องดื่ม ได้แก่ รูปแบบเครื่องดื่ม (เย็น ร้อน ปั่น) ระดับการคั่วเมล็ดกาแฟ ระดับความหวาน (125%, 100%, 75%, 50%, 25%, 0%) และ Add-ons เป็น «extend» ของ UC-07 ใช้เมื่อสินค้าเป็นเครื่องดื่ม
Pre-conditions	ผู้ใช้อยู่ในหน้าต่างตัวเลือกของเครื่องดื่ม
Post-conditions	ตัวเลือกที่เลือกถูกรวมในรายการ และราคาต่อหน่วยเท่ากับราคารูฐาน + ราคารูปแบบเครื่องดื่ม + ราคา Add-ons
Main Success Scenario	1. ระบบแสดงเฉพาะตัวเลือกที่เปิดใช้งานของสินค้านั้น
Extensions	• 4a. ผู้ใช้ไม่เลือก Add-on ระบบแสดงว่า "ไม่มีเพิ่มเติม"
หมายเหตุ	Add-on ที่ปิดใช้งานจะไม่แสดงให้เลือก

UC-09: Apply Promotion

Actor	Cashier (Admin สิบบทสดสิทธิ์)
Description	ใช้โปรโมชั่นกับออเดอร์เพื่อลดราคา เป็น «extend» ของ UC-05 ใช้เมื่อมีโปรโมชั่นที่เข้าเงื่อนไข
Pre-conditions	มีออเดอร์ที่มีรายการสินค้า และมีโปรโมชั่นที่เปิดใช้งานอยู่
Post-conditions	ออเดอร์มีส่วนลดและยอดสุทธิถูกคำนวณใหม่
Main Success Scenario	1. ผู้ใช้กดใช้โปรโมชั่น
Extensions	• 3a. ออเดอร์ไม่เข้าเงื่อนไขของโปรโมชั่น ระบบแจ้งเหตุผลและไม่ใช้ส่วนลด • 4a. ผู้ใช้ยกเลิกโปรโมชั่น ระบบคืนยอดเดิม

UC-10: Edit / Remove Order Item

Actor	Cashier (Admin สิบบทสดสิทธิ์)
Description	แก้ไขหรือลบรายการสินค้าที่อยู่ในออเดอร์ก่อนชำระเงิน
Pre-conditions	ออเดอร์ยังไม่ชำระเงินและมีรายการสินค้า
Post-conditions	รายการในออเดอร์ถูกแก้ไขหรือลบ และยอดรวมถูกคำนวณใหม่
Main Success Scenario	1. ผู้ใช้เลือกรายการในออเดอร์
Extensions	• 4a. ลบรายการสุดท้าย ออเดอร์เหลือว่างและไม่สามารถชำระเงินได้
หมายเหตุ	ออเดอร์ที่ชำระเงินแล้วไม่สามารถแก้ไขได้

UC-11: Cancel Order

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	ยกเลิกออเดอร์ทั้งหมดที่ยังไม่ชำระเงิน
Pre-conditions	ออเดอร์ยังไม่ชำระเงิน
Post-conditions	ออเดอร์มีสถานะยกเลิก (CANCELLED) และรายการในหน้าสั่งซื้อถูกล้าง
Main Success Scenario	1. ผู้ใช้กดยกเลิกออเดอร์
Extensions	• 3a. ผู้ใช้ไม่ยืนยัน ระบบคงออเดอร์เดิม

UC-12: Process Payment

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	รับชำระเงินสำหรับออเดอร์ มีความสัมพันธ์ «include» กับ UC-13 และ UC-14
Pre-conditions	ออเดอร์มีรายการสินค้าและมียอดสุทธิมากกว่า 0
Post-conditions	ออเดอร์ถูกบันทึกเป็นชำระแล้ว (PAID) และปรากฏในประวัติออเดอร์
Main Success Scenario	1. ผู้ใช้กดชำระเงิน
Extensions	• 3a. เงินสดที่รับน้อยกว่ายอดสุทธิ ระบบไม่อนุญาตให้ยืนยัน • 3b. ชำระ QR ไม่สำเร็จหรือหมดเวลา ระบบแจ้งเตือนและให้เลือกวิธีชำระใหม่หรือยกเลิก

UC-13: Select Payment Method (Cash / QR)

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	เลือกวิธีชำระเงินระหว่างเงินสดหรือ QR เป็นส่วนหนึ่งของ UC-12 («include»)
Pre-conditions	ผู้ใช้อยู่ในขั้นตอนการชำระเงิน
Post-conditions	ระบบทราบวิธีชำระเงินที่ใช้กับออเดอร์นี้
Main Success Scenario	1. ระบบแสดงตัวเลือกวิธีชำระเงิน (เงินสด / QR)
Main Success Scenario	1. ระบบแสดงตัวเลือกวิธีชำระเงิน (เงินสด / QR) 2. ผู้ใช้เลือกวิธีชำระเงิน 3. ระบบบันทึกวิธีที่เลือกและแสดงหน้าจอที่เกี่ยวข้อง
Extensions	• 2a. ผู้ใช้เปลี่ยนวิธีชำระเงินก่อนยืนยัน ระบบเปลี่ยนไปหน้าจอของวิธีใหม่

UC-14: View / Print Receipt

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	แสดงและพิมพ์ใบเสร็จหลังชำระเงิน มีเลขที่ออเดอร์ วันที่เวลา ชื่อพนักงาน รายการสินค้า พร้อมตัวเลือก ส่วนลด ยอดรวม และวิธีชำระเงิน เป็นส่วนหนึ่งของ UC-12 («include»)
Pre-conditions	การชำระเงินสำเร็จ
Post-conditions	ใบเสร็จถูกแสดงบนหน้าจอและ/หรือพิมพ์ออกมา
Main Success Scenario	1. 1. ระบบสร้างใบเสร็จจากข้อมูลออเดอร์
Main Success Scenario	• 1. ระบบสร้างใบเสร็จจากข้อมูลออเดอร์ 2. ระบบแสดงใบเสร็จบนหน้าจอ 3. ผู้ใช้กดพิมพ์ 4. ระบบส่งใบเสร็จไปยังเครื่องพิมพ์

Extensions	• 4a. ไม่พบเครื่องพิมพ์หรือพิมพ์ไม่สำเร็จ ระบบแจ้งเตือนและให้ดูใบเสร็จจบหน้าจอหรือพิมพ์ซ้ำภายหลัง
------------	---

UC-15: Search Order History

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	ค้นหาออเดอร์ย้อนหลังด้วยเลขที่ออเดอร์ วันที่ สถานะ หรือวิธีชำระเงิน มีความสัมพันธ์ «include» กับ UC-16
Pre-conditions	ผู้ใช้เข้าสู่ระบบอยู่
Post-conditions	ผู้ใช้เห็นรายการออเดอร์ตามเงื่อนไขที่ค้นหา
Main Success Scenario	1. ผู้ใช้เปิดหน้าประวัติออเดอร์
Extensions	• 3a. ไม่พบออเดอร์ ระบบแจ้งว่าไม่พบข้อมูล

UC-16: View Order Detail

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	แสดงรายละเอียดออเดอร์ ได้แก่ รายการสินค้า ตัวเลือก โน้ต ส่วนลด ยอดรวม วิธีชำระเงิน เวลา และพนักงานที่ทำรายการ เป็นส่วนหนึ่งของ UC-15 และ UC-17 («include»)
Pre-conditions	มีออเดอร์ในระบบ
Post-conditions	ผู้ใช้เห็นข้อมูลออเดอร์ครบถ้วน
Main Success Scenario	1. ผู้ใช้เลือกออเดอร์
Extensions	• 2a. ไม่พบข้อมูลออเดอร์ ระบบแจ้งข้อผิดพลาด

UC-17: Reprint Receipt

Actor	Cashier (Admin สืบทอดสิทธิ์)
Description	พิมพ์ใบเสร็จซ้ำของออเดอร์ที่ชำระแล้ว มีความสัมพันธ์ «include» กับ UC-16
Pre-conditions	ออเดอร์ที่เลือกชำระเงินแล้ว
Post-conditions	ใบเสร็จฉบับสำเนาถูกพิมพ์
Main Success Scenario	1. ผู้ใช้ค้นหาและเลือกออเดอร์
Extensions	• 3a. ออเดอร์ถูกยกเลิกหรือยังไม่ชำระ ระบบไม่อนุญาตให้พิมพ์ • 4a. พิมพ์ไม่สำเร็จ ระบบแจ้งเตือนให้ลองใหม่

UC-18: Manage Products

Actor	Admin
Description	เพิ่ม แก้ไข ลบ หรือปิดใช้งานสินค้า (ชื่อ ราคา หมวดหมู่ รูปภาพ และตัวเลือกของสินค้า)
Pre-conditions	Admin เข้าสู่ระบบแล้ว
Post-conditions	ข้อมูลสินค้าถูกอัปเดตและมีผลกับหน้าสั่งซื้อ
Main Success Scenario	1. Admin เปิดหน้าจัดการสินค้า
Extensions	• 2a. ลบสินค้าที่เคยอยู่ในออเดอร์ ระบบแนะนำให้ปิดใช้งานแทนการลบ • 5a. ข้อมูลไม่ครบหรือราคาไม่ถูกต้อง ระบบแจ้งเตือนและไม่บันทึก

UC-19: Manage Categories

Actor	Admin
Description	เพิ่ม แก้ไข หรือลบหมวดหมู่สินค้า (เช่น กาแฟ ชา ขนม)
Pre-conditions	Admin เข้าสู่ระบบแล้ว
Post-conditions	หมวดหมู่ถูกอัปเดตและแสดงในหน้าสั่งซื้อ
Main Success Scenario	1. Admin เปิดหน้าจัดการหมวดหมู่
Main Success Scenario	1. Admin เปิดหน้าจัดการหมวดหมู่ 2. Admin เลือกเพิ่มหรือแก้ไขหมวดหมู่ 3. Admin กรอกชื่อหมวดหมู่ 4. Admin กดบันทึก 5. ระบบตรวจสอบและบันทึก
Extensions	• 2a. ลบหมวดหมู่ที่ยังมีสินค้า ระบบไม่อนุญาต ต้องย้ายสินค้าออกก่อน • 5a. ชื่อหมวดหมู่ซ้ำหรือว่าง ระบบแจ้งเตือน

UC-20: Manage Add-ons

Actor	Admin
Description	จัดการที่อปปิ้งส่วนกลาง (Add-ons) ได้แก่ สร้าง แก้ไข ลบ และเปิด/ปิดใช้งาน โดยมีชื่อคำอธิบายย่อ และราคาเพิ่ม รวมถึงค้นหาและกรองตามหมวดหมู่ (ทั้งหมด / กาแฟ / ชา)
Pre-conditions	Admin เข้าสู่ระบบแล้ว
Post-conditions	รายการ Add-on ถูกอัปเดต และการเปลี่ยนแปลงมีผลกับทุกเมนูที่ใช้ Add-on นั้น
Main Success Scenario	1. Admin เปิดหน้าจัดการ Add-ons
Extensions	• 4a. Admin ยกเลิกการลบ ระบบคงข้อมูลเดิม • 5a. ไม่กรอกชื่อ ชื่อซ้ำ หรือราคาติดลบ ระบบแจ้งเตือนและไม่บันทึก

UC-21: Link Add-ons to Product

Actor	Admin
Description	เลือกว่า Add-on ส่วนกลางตัวใดจะนำไปใช้กับสินค้าแต่ละรายการ
Pre-conditions	มี Add-on และสินค้าอยู่ในระบบ
Post-conditions	สินค้าถูกผูกกับ Add-on ที่เลือก และ Add-on จะปรากฏในหน้าต่างตัวเลือกเมื่อสั่งสินค้านั้น (UC-08)
Main Success Scenario	1. Admin เปิดหน้าแก้ไขสินค้า 2. ระบบแสดงรายการ Add-on ทั้งหมด 3. Admin เลือกหรือยกเลิกการเลือก Add-on ที่ต้องการ 4. Admin กดบันทึก 5. ระบบบันทึกการผูกข้อมูลกับสินค้า
Extensions	• 3a. Admin เลือกสร้าง Add-on ใหม่จากหน้านี้ ระบบเปิดฟอร์มสร้าง Add-on โดยระบุชื่อเมนูที่กำลังแก้ไข
หมายเหตุ	Add-on ที่ปิดใช้งานจะไม่แสดงในหน้าสั่งซื้อ แม้จะผูกไว้

UC-22: Manage Promotions

Actor	Admin
Description	สร้าง แก้ไข ลบ และเปิด/ปิดโปรโมชั่น (ชื่อ ประเภทส่วนลด เงื่อนไข และช่วงเวลา)
Pre-conditions	Admin เข้าสู่ระบบแล้ว
Post-conditions	โปรโมชั่นถูกอัปเดตและพร้อมให้เลือกใช้ใน UC-09
Main Success Scenario	1. Admin เปิดหน้าจัดการโปรโมชั่น 2. Admin เลือกสร้างหรือแก้ไขโปรโมชั่น 3. Admin กรอกรายละเอียดและกำหนดช่วงเวลา 4. Admin กดบันทึก 5. ระบบตรวจสอบและบันทึก
Extensions	• 5a. ช่วงวันที่ไม่ถูกต้องหรือข้อมูลไม่ครบ ระบบแจ้งเตือนและไม่บันทึก

UC-23: View Sales Reports

Actor	Admin
Description	ดูรายงานยอดขายตามช่วงเวลา เช่น ยอดขายรวม จำนวนออเดอร์ สินค้าขายดี และยอดแยกตามวิธีชำระเงิน
Pre-conditions	Admin เข้าสู่ระบบแล้ว
Post-conditions	Admin เห็นข้อมูลสรุปยอดขายตามที่เลือก
Main Success Scenario	1. Admin เปิดหน้ารายงานยอดขาย 2. Admin เลือกช่วงเวลา (รายวัน / รายเดือน / กำหนดเอง) 3. ระบบรวบรวมข้อมูลออเดอร์ที่ชำระแล้วในช่วงนั้น 4. ระบบแสดงสรุปและรายละเอียดของรายงาน
Extensions	• 4a. ไม่มีข้อมูลในช่วงเวลาที่เลือก ระบบแจ้งว่าไม่มีข้อมูล

UC-24: Manage Users & Passwords

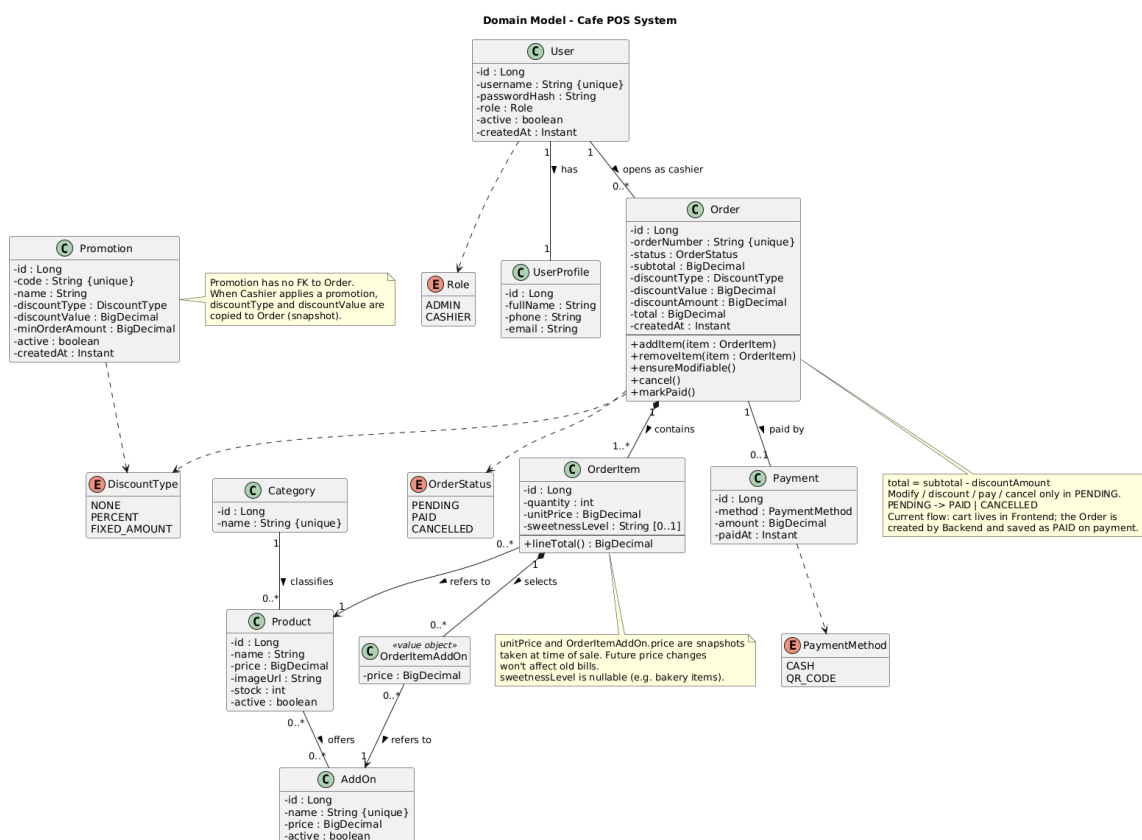
Actor	Admin
Description	เพิ่ม แก้ไข ปิดใช้งานบัญชีผู้ใช้ กำหนดบทบาท (Cashier / Admin) และรีเซ็ตรหัสผ่านให้ผู้ใช้
Pre-conditions	Admin เข้าสู่ระบบแล้ว
Post-conditions	ข้อมูลบัญชีผู้ใช้หรือรหัสผ่านถูกอัปเดต
Main Success Scenario	1. Admin เปิดหน้าจัดการผู้ใช้ 2. Admin เลือกเพิ่มผู้ใช้ แก้ไขผู้ใช้ หรือรีเซ็ตรหัสผ่าน 3. Admin กรอกข้อมูลที่จำเป็น เช่น ชื่อผู้ใช้ ชื่อ-นามสกุล บทบาท และรหัสผ่านชั่วคราว 4. Admin กดบันทึก 5. ระบบตรวจสอบ เข้ารหัสรหัสผ่าน และบันทึกข้อมูล
Extensions	• 2a. Admin พยายามปิดใช้งานบัญชีของตนเอง ระบบไม่อนุญาต • 5a. ชื่อผู้ใช้ซ้ำหรือข้อมูลไม่ครบ ระบบแจ้งเตือนและไม่บันทึก

3.2 Domain Model

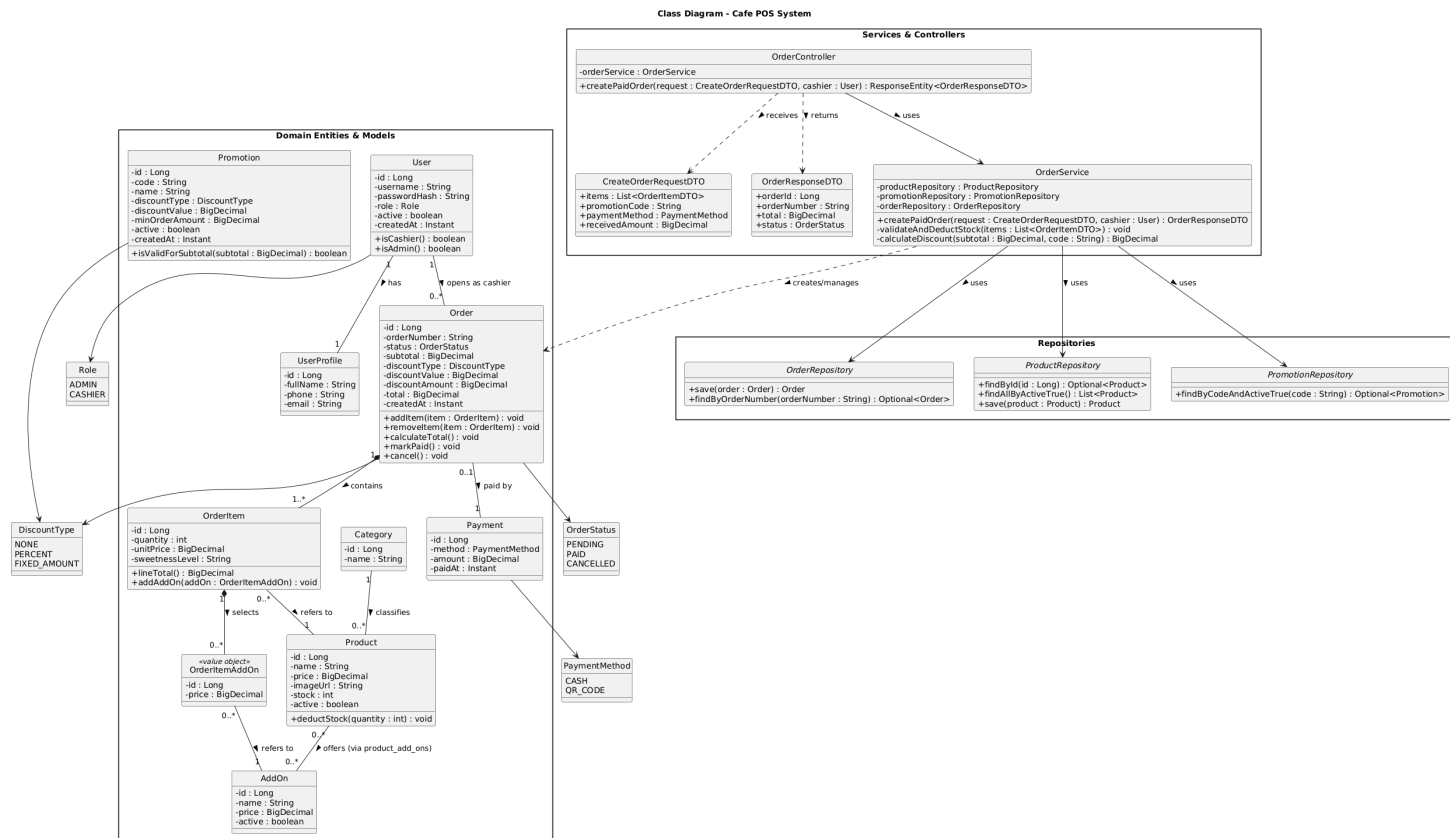
Domain Model แสดงแนวคิดหลักของระบบและความสัมพันธ์ ได้แก่ User (มี UserProfile 1:1) เปิด Order ได้หลายรายการ Order ประกอบด้วย OrderItem ตั้งแต่ 1 รายการ แต่ละ OrderItem อ้างถึง Product และเลือก Add-on ได้ (OrderItemAddOn เป็น Value Object ที่เก็บราคา Add-on ณ เวลาขาย) Product อยู่ใน Category เดียวและมี Add-on ได้หลายตัว (Many-to-Many) Order มี Payment ได้ไม่เกิน 1 รายการ ส่วน Promotion เป็นข้อมูลอิสระที่ไม่มีความสัมพันธ์กับ Order โดยตรง เมื่อใช้โปรโมชั่น ระบบคัดลอกชนิดและมูลค่าส่วนลดมาเก็บใน Order แทน (Snapshot) ภาพที่ 3.3 อยู่ในหน้าถัดไป

3.3 Class Diagram และ Design Patterns

ภาพที่ 3.4 แสดงคลาสที่เกี่ยวข้องกับ Design Patterns หลักของระบบ ได้แก่ Strategy (DiscountStrategy), State (OrderState) และ Observer (OrderPaidEvent กับ SaleAuditLogListener) รายละเอียดอยู่ในบทที่ 4

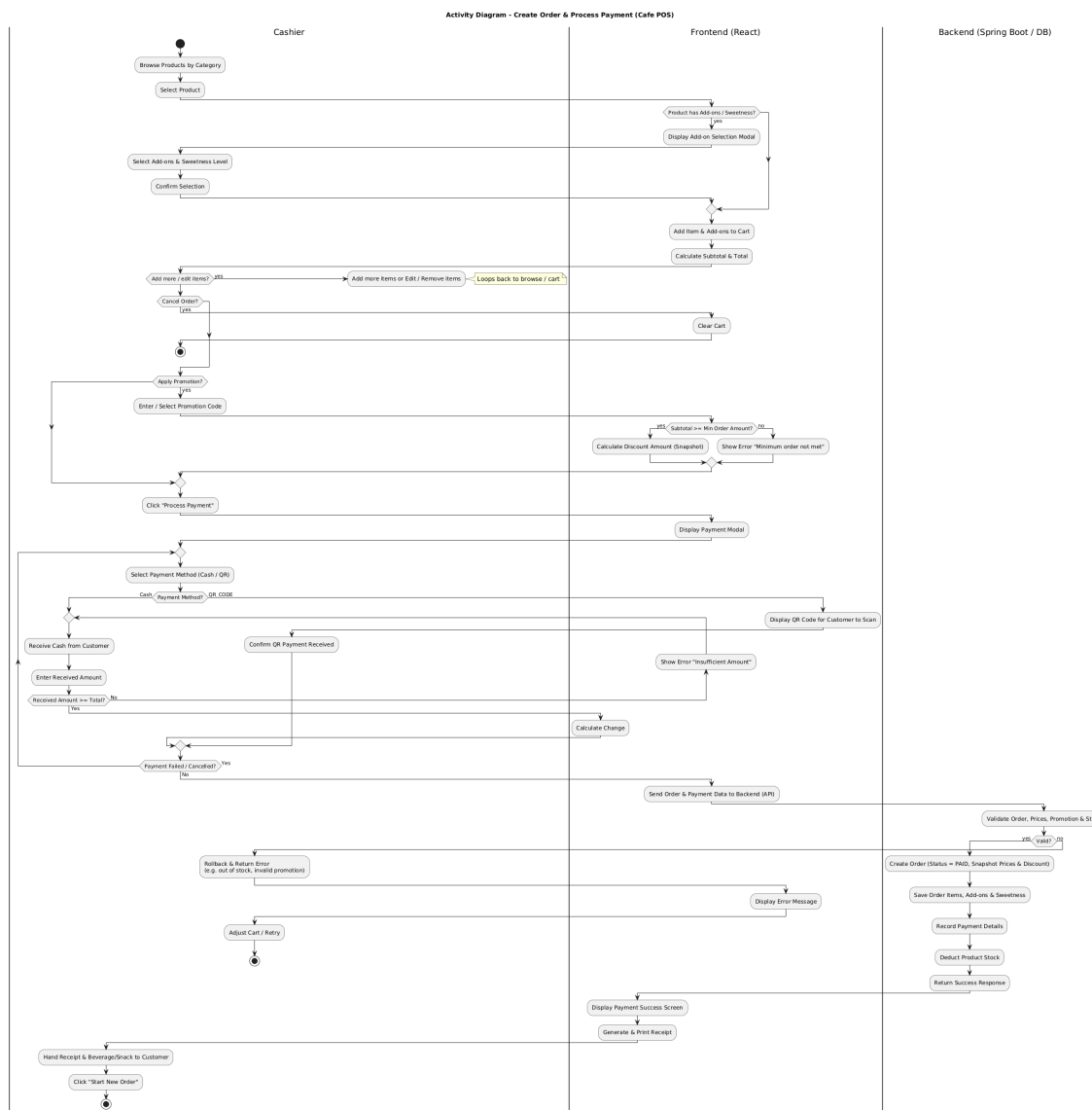


ภาพที่ 3.2 Domain Model



ภาพที่ 3.3 Class Diagram ของส่วน Order, Payment และ Design Patterns

3.4 Activity Diagram

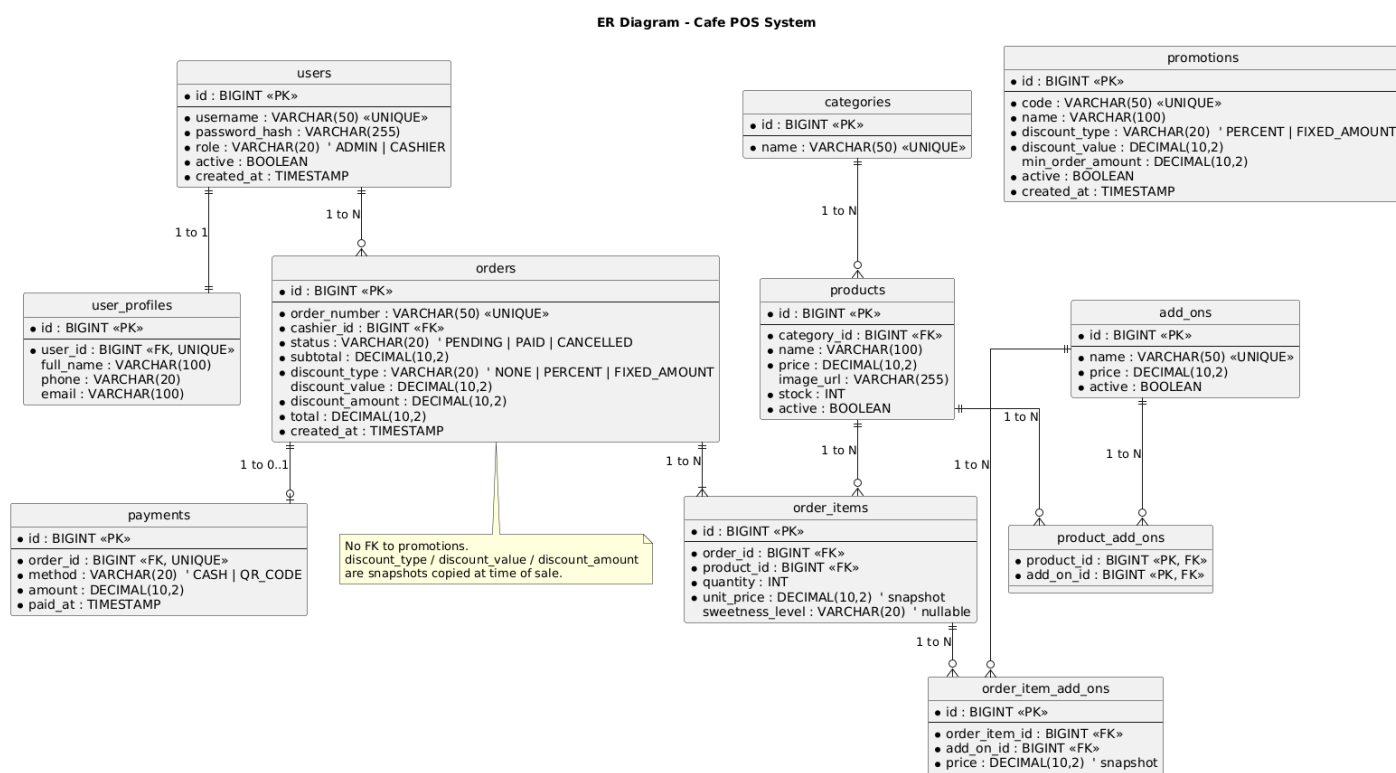


ภาพที่ 3.4 Activity Diagram: สร้างออเดอร์และชำระเงิน

ภาพที่ 3.4 แสดงลำดับกิจกรรมตั้งแต่เลือกสินค้า สร้างออเดอร์ ใช้ส่วนลด จนถึงชำระเงิน โดยแบ่งช่องตามผู้ปฏิบัติ (Cashier, Frontend, Backend) ทุกขั้นตอนที่เปลี่ยนข้อมูลต้องผ่านการตรวจสอบที่ Backend และออเดอร์ที่ไม่ได้อยู่ในสถานะ PENDING จะถูกปฏิเสธด้วย 409 Conflict

3.5 Entity Relationship Diagram และ Data Dictionary

ฐานข้อมูลมี 11 ตาราง สร้างและเปลี่ยนแปลงผ่าน Flyway Migration V1-V5 ความสัมพันธ์ประกอบด้วย One-to-One (users-user_profiles, orders-payments), One-to-Many (categories-products, users-orders, orders-order_items, order_items-order_item_add_ons) และ Many-to-Many (products-add_ons ผ่าน product_add_ons และ order_items-add_ons ผ่าน order_item_add_ons) ตาราง promotions เป็นตารางอิสระ



ภาพที่ 3.5 ER Diagram

Data Dictionary

ตารางที่ 3.2 ตาราง users (ผู้ใช้งานระบบ)

Column	Type	Constraint	Default	คำอธิบาย
id	BIGINT	PK, IDENTITY		รหัสผู้ใช้
username	VARCHAR(50)	NN, UK		ชื่อสำหรับเข้าสู่ระบบ
password_hash	VARCHAR(255)	NN		รหัสผ่านที่ hash แล้ว (ห้ามเก็บ plain text)
role	VARCHAR(20)	NN, CHECK IN (ADMIN, CASHIER)		สิทธิ์ผู้ใช้ — enum Role
active	BOOLEAN	NN	TRUE	false = ระงับการใช้งาน
created_at	TIMESTAMPTZ(6)	NN		วันที่เวลาที่สร้าง (@CreationTimestamp)

ตารางที่ 3.3 ตาราง user_profiles (ข้อมูลส่วนตัวของผู้ใช้ (1:1 กับ `users`))

Column	Type	Constraint	Default	คำอธิบาย
user_id	BIGINT	PK, FK → users.id ON DELETE CASCADE		ใช้ PK ร่วมกับ users (@MapsId)
full_name	VARCHAR(100)	NN		ชื่อ-นามสกุล
phone	VARCHAR(20)			เบอร์โทร
email	VARCHAR(100)			อีเมล

ตารางที่ 3.4 ตาราง categories (หมวดหมู่สินค้า)

Column	Type	Constraint	Default	คำอธิบาย
id	BIGINT	PK, IDENTITY		รหัสหมวด
name	VARCHAR(50)	NN, UK		ชื่อหมวด เช่น Coffee, Tea, Bakery

ตารางที่ 3.5 ตาราง products (สินค้า/เมนู)

Column	Type	Constraint	Default	คำอธิบาย
id	BIGINT	PK, IDENTITY		รหัสสินค้า
category_id	BIGINT	NN, FK → categories.id		หมวดของสินค้า
name	VARCHAR(100)	NN		ชื่อเมนู
price	NUMERIC(10,2)	NN, CHECK ≥ 0		ราคาขายปัจจุบัน (บาท)
image_url	TEXT			URL รูปสินค้า (เปลี่ยนจาก VARCHAR(500) → TEXT ใน V6)
active	BOOLEAN	NN	TRUE	soft delete — false = ไม่แสดงในหน้าขาย

ตารางที่ 3.6 ตาราง add_ons (ตัวเลือกเพิ่มเติม)

Column	Type	Constraint	Default	คำอธิบาย
id	BIGINT	PK, IDENTITY		รหัส add-on
name	VARCHAR(50)	NN, UK		เช่น Extra Shot, Oat Milk
price	NUMERIC(10,2)	NN, CHECK ≥ 0		ราคาต่อบวกเพิ่ม (บาท)
active	BOOLEAN	NN	TRUE	false = ปิดใช้งาน

ตารางที่ 3.7 ตาราง product_add_ons (add-on ที่เลือกได้ของแต่ละสินค้า (M:N))

Column	Type	Constraint	Default	คำอธิบาย
product_id	BIGINT	PK, FK $\rightarrow$ products.id ON DELETE CASCADE		สินค้า
add_on_id	BIGINT	PK, FK $\rightarrow$ add_ons.id ON DELETE CASCADE		add-on

ตารางที่ 3.8 ตาราง orders (ออเดอร์/บิลขาย)

Column	Type	Constraint	Default	คำอธิบาย
id	BIGINT	PK, IDENTITY		รหัสออเดอร์
order_number	VARCHAR(20)	NN, UK		เลขที่บิลที่แสดงบนใบเสร็จ
cashier_id	BIGINT	NN, FK $\rightarrow$ users.id		แคชเชียร์ที่เปิดบิล
status	VARCHAR(20)	NN, CHECK IN (PENDING, PAID, CANCELLED)	PENDING (ใน entity)	สถานะ — enum OrderStatus

Column	Type	Constraint	Default	คำอธิบาย
subtotal	NUMERIC(10,2)	NN, CHECK ≥ 0		ผลรวมราคาทุกรายการ (รวมค่า add-on) ก่อนส่วนลด — ดูสูตรที่ order_items
discount_type	VARCHAR(20)	NN, CHECK IN (NONE, PERCENT, FIXED_AMOUNT)	NONE	แบบส่วนลด — enum DiscountType (เลือก DiscountStrategy)
discount_value	NUMERIC(10,2)	CHECK ≥ 0		ค่าที่แคชเชียร์กรอก: PERCENT = เปอร์เซ็นต์ (10 = 10%), FIXED_AMOUNT = บาท, NONE = ว่าง
discount_amount	NUMERIC(10,2)	NN, CHECK ≥ 0	0	จำนวนเงินที่ลดจริง คำนวณจาก discount_type + discount_value + subtotal
total	NUMERIC(10,2)	NN, CHECK ≥ 0		ยอดสุทธิ = subtotal - discount_amount
created_at	TIMESTAMPTZ(6)	NN		วันเวลาที่เปิดบิล

ตารางที่ 3.9 ตาราง order_items (รายการสินค้าในออเดอร์)

Column	Type	Constraint	Default	คำอธิบาย
id	BIGINT	PK, IDENTITY		รหัสรายการ
order_id	BIGINT	NN, FK → orders.id ON DELETE CASCADE		ออเดอร์ที่รายการนี้อยู่
product_id	BIGINT	NN, FK → products.id		สินค้าที่สั่ง
quantity	INT	NN, CHECK > 0		จำนวน
unit_price	NUMERIC(10,2)	NN, CHECK ≥ 0		ราคาสินค้าต่อหน่วย ณ เวลาที่สั่ง (snapshot) ไม่รวม add-on

ตารางที่ 3.10 ตาราง order_item_add_ons (add-on ที่เลือกในแต่ละรายการของบิล (M:N พร้อมข้อมูล))

Column	Type	Constraint	Default	คำอธิบาย
order_item_id	BIGINT	PK, FK → order_items.id ON DELETE CASCADE		รายการในบิล
add_on_id	BIGINT	PK, FK → add_ons.id		add-on ที่เลือก
price	NUMERIC(10,2)	NN, CHECK ≥ 0		ราคา add-on ณ เวลาที่สั่ง (snapshot) ต่อ 1 หน่วยสินค้า

ตารางที่ 3.11 ตาราง promotions (โปรโมชั่น/ส่วนลด (ไม่มี FK ไปที่ orders))

Column	Type	Constraint	Default	คำอธิบาย
id	BIGINT	PK, IDENTITY		รหัสโปรโมชั่น
code	VARCHAR(50)	NN, UK		รหัสคูปอง เช่น MEMBER10, WELCOME20
name	VARCHAR(100)	NN		ชื่อโปรโมชั่นที่แสดงบน UI
discount_type	VARCHAR(20)	NN, CHECK IN (PERCENT, FIXED_AMOUNT)		แบบส่วนลด — enum DiscountType (ไม่มี NONE ใน promotions)
discount_value	NUMERIC(10,2)	NN, CHECK ≥ 0		ค่าส่วนลด: PERCENT = เปอร์เซ็นต์, FIXED_AMOUNT = บาท
min_order_amount	NUMERIC(10,2)	CHECK ≥ 0 หรือ NULL		ยอดขั้นต่ำที่ใช้โปรโมชั่น ได้ — NULL = ไม่มีขั้นต่ำ
active	BOOLEAN	NN	TRUE	false = ปิดโปรโมชั่น
created_at	TIMESTAMPTZ	NN	CURRENT_TIMESTAMP	วันที่เวลาที่สร้าง

ตารางที่ 3.12 ตาราง payments (การชำระเงิน (1:1 กับ `orders`))

Column	Type	Constraint	Default	คำอธิบาย
id	BIGINT	PK, IDENTITY		รหัสการชำระ
order_id	BIGINT	NN, UK, FK → orders.id		ออเดอร์ที่ชำระ (UNIQUE = 1 ออเดอร์ 1 payment)
method	VARCHAR(20)	NN, CHECK IN (CASH, QR_CODE, CARD)		ช่องทาง — enum PaymentMethod
amount	NUMERIC(10,2)	NN, CHECK ≥ 0		จำนวนเงินที่ชำระ
paid_at	TIMESTAMPTZ(6)	NN		วันเวลาที่ชำระ

หมายเหตุ: PK = Primary Key, FK = Foreign Key, UK = Unique, NN = Not Null

3.6 Sequence Diagram

ภาพที่ 3.6 แสดงลำดับการทำงานของสถานการณ์ Create Order & Process Payment ตั้งแต่พนักงานขายเลือกสินค้าจนถึงชำระเงินและพิมพ์ใบเสร็จ โดยมีผู้เข้าร่วมคือ Cashier, React Frontend, OrderController, OrderService, ProductRepository, PromotionRepository และ Database แบ่งการทำงานเป็น 3 ช่วง ดังนี้

3.6.1 ช่วงที่ 1 Build Cart (ทำงานที่ Frontend เท่านั้น) Cashier เลือกสินค้าจากหมวดหมู่หรือค้นหา เมื่อสินค้ามี Add-on ระบบแสดงหน้าต่างให้เลือก Add-on และระดับความหวาน จากนั้น Frontend เพิ่มรายการลงตะกร้าและคำนวณ Subtotal หากต้องการใช้โปรโมชั่น Cashier กรอกรหัสโปรโมชั่น ระบบตรวจสอบยอดสั่งซื้อขั้นต่ำและคำนวณส่วนลด (Snapshot) ในขั้นตอนนี้ยังไม่มีเรียก Backend

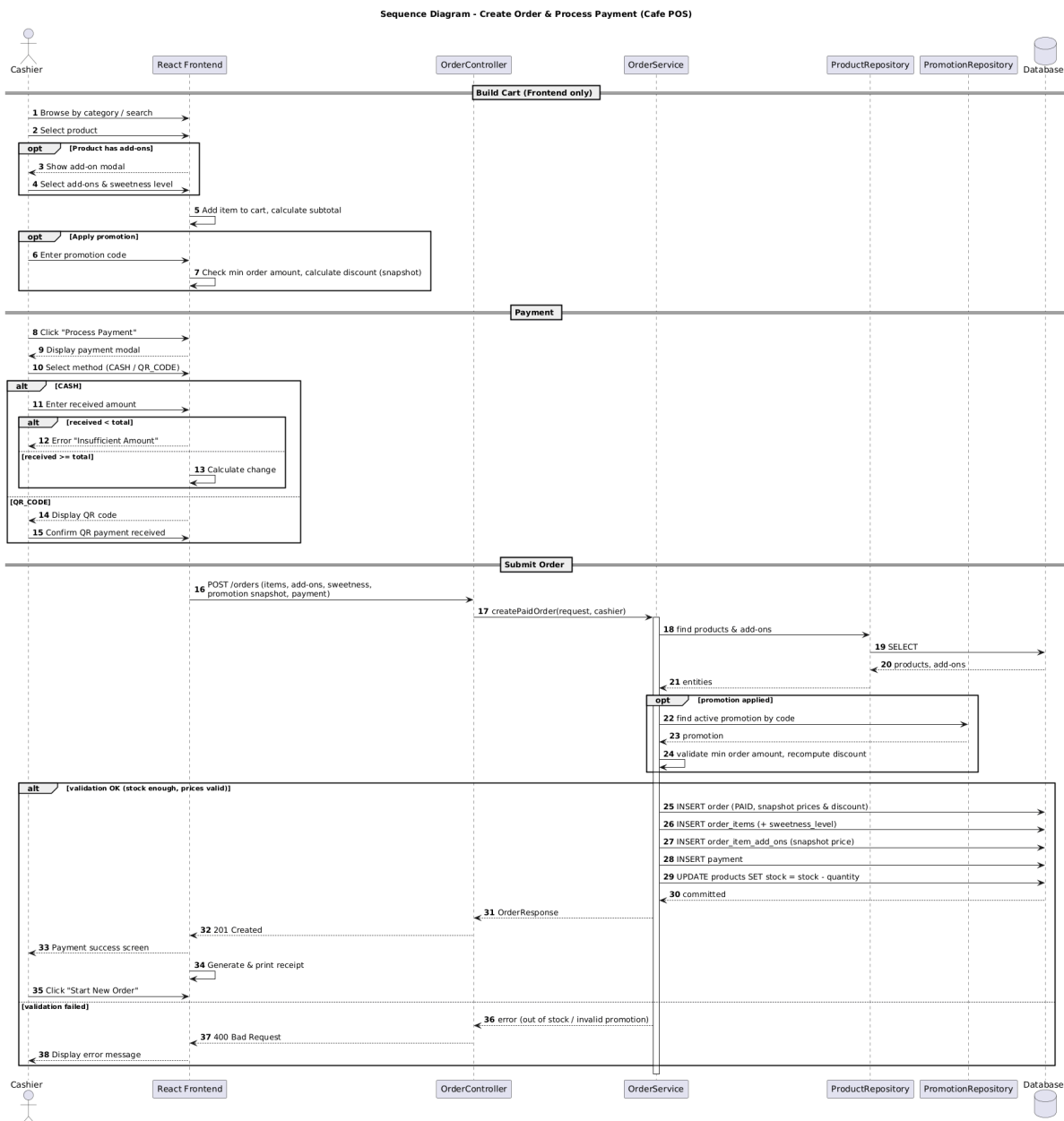
3.6.2 ช่วงที่ 2 Payment เมื่อ Cashier กด “Process Payment” ระบบแสดงหน้าต่างชำระเงินให้เลือกวิธีชำระเงินได้แก่ CASH หรือ QR_CODE

1. กรณี CASH: Cashier กรอกจำนวนเงินที่รับ หากน้อยกว่ายอดสุทธิ ระบบแสดงข้อผิดพลาด “Insufficient Amount” หากเท่ากับหรือมากกว่า ระบบคำนวณเงินทอน

2. กรณี QR_CODE: ระบบแสดง QR Code และรอยืนยันว่าได้รับเงินแล้ว

3.6.3 ช่วงที่ 3 Submit Order หลังชำระเงิน Frontend ส่ง POST /orders พร้อมรายการสินค้า Add-on ระดับความหวาน ข้อมูลโปรโมชั่น และข้อมูลการชำระเงิน ไปยัง OrderController ซึ่งเรียก createPaidOrder (request, cashier) ที่ OrderService จากนั้น OrderService ทำงานตามลำดับดังนี้

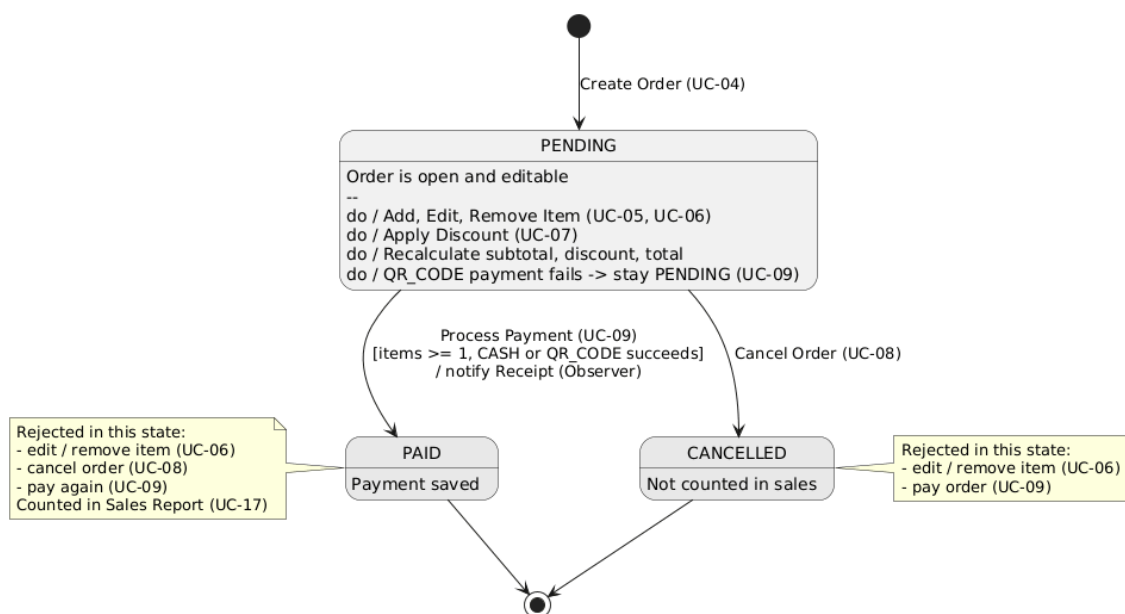
1. ดึงข้อมูลสินค้าและ Add-on จาก ProductRepository (SELECT จาก Database)
2. ถ้ามีการใช้โปรโมชั่น ค้นหาโปรโมชั่นที่เปิดใช้งานจากรหัสผ่าน PromotionRepository ตรวจสอบยอดขั้นต่ำ และคำนวณส่วนลดใหม่
3. ตรวจสอบความถูกต้อง (สต็อกเพียงพอ และราคาถูกต้อง) ผลลัพธ์แยกเป็น 2 กรณี
 - **ตรวจสอบผ่าน:** ระบบบันทึกข้อมูลภายใน Transaction เดียว ได้แก่ INSERT order (สถานะ PAID พร้อมราคาและส่วนลดแบบ Snapshot), INSERT order_items (พร้อมระดับความหวาน), INSERT order_item_add_ons (ราคา Snapshot), INSERT payment และ UPDATE products ลดจำนวนสต็อก แล้ว Commit จากนั้นส่ง OrderResponse กลับผ่าน Controller เป็น 201 Created Frontend แสดงหน้าจอชำระเงินสำเร็จ สร้างและพิมพ์ใบเสร็จ และ Cashier กด “Start New Order” เพื่อเริ่มบิลใหม่
 - **ตรวจสอบไม่ผ่าน** (สินค้าหมดสต็อกหรือโปรโมชั่นไม่ถูกต้อง): ระบบส่ง 400 Bad Request กลับมา และ Frontend แสดงข้อความแจ้งข้อผิดพลาด โดยไม่มีการบันทึกข้อมูลใด ๆ



ภาพที่ 3.6 Sequence Diagram: Create Order & Process Payment

3.7 State Diagram

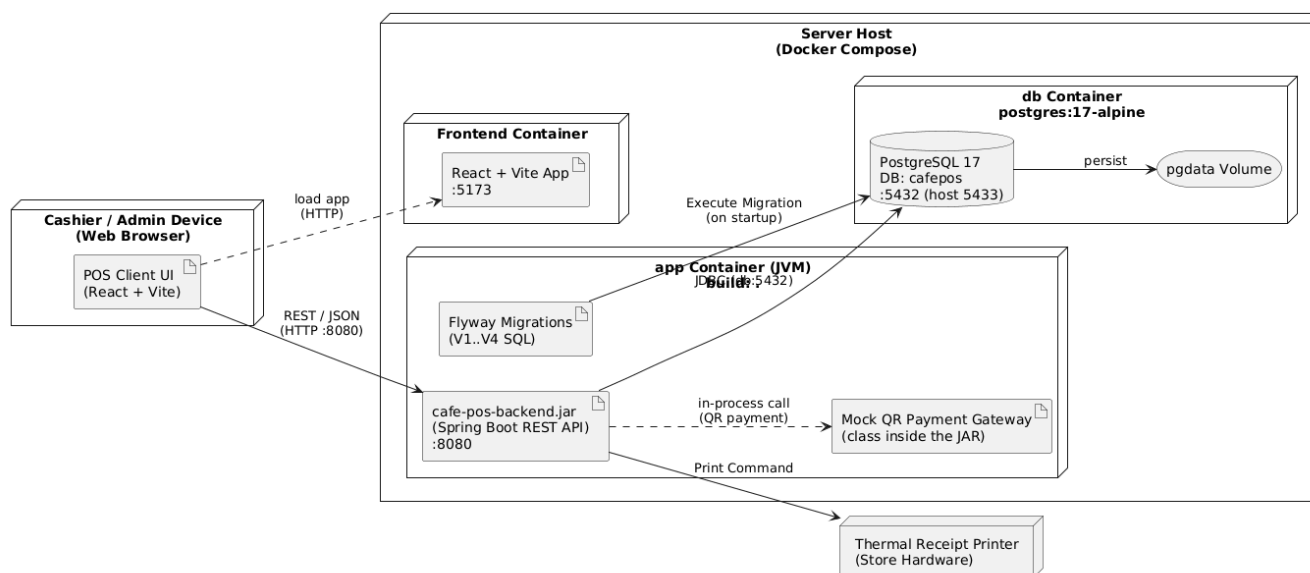
ออเดอร์เริ่มที่สถานะ PENDING และเปลี่ยนเป็น PAID เมื่อชำระเงิน หรือ CANCELLED เมื่อยกเลิก
เมื่อออกจาก PENDING แล้วไม่สามารถแก้ไข ชำระ หรือยกเลิกซ้ำได้ ระบบจะตอบ 409 Conflict



ภาพที่ 3.7 State Diagram

3.8 Deployment Diagram

ในการใช้งานจริง Frontend ที่ build แล้วถูกคัดลอกเข้า Spring Boot เป็น Static Files และ Deploy เป็น Docker image เดียวบน Render (Same Origin จึงไม่ต้องตั้งค่า CORS) เชื่อมต่อฐานข้อมูล PostgreSQL บน Neon ผ่านตัวแปรสภาพแวดล้อม DB_URL, DB_USERNAME, DB_PASSWORD ส่วนการพัฒนาในเครื่องใช้ Docker Compose รัน PostgreSQL และ Application



ภาพที่ 3.8 Deployment Diagram

บทที่ 4

การประยุกต์ใช้หลักการออกแบบและ Design Patterns

4.1 รายงานการวิเคราะห์ SOLID Principles

ตารางที่ 4.1 แสดงตัวอย่างโค้ดจริงที่สะท้อนหลัก SOLID แต่ละข้อ

ตารางที่ 4.1 การประยุกต์หลัก SOLID

หลักการ	ไฟล์ / คลาส	บรรทัด	เหตุผลและผลลัพธ์
S Single Responsibility	OrderServiceImpl, OrderMapper, domain/state/*, service/discount/*	OrderServiceImpl L35-228	Service ดูแลลำดับงานของออเดอร์เท่านั้น การแปลง Entity<-->DTO อยู่ที่ Mapper กฎของสถานะอยู่ใน State และการคำนวณส่วนลดอยู่ใน Strategy แต่ละคลาสมีเหตุผลให้เปลี่ยนเพียงข้อเดียว
O Open/Closed	DiscountStrategy และ NoDiscount, PercentDiscount, FixedAmountDiscount	DiscountStrategy L1-22; OrderServiceImpl L53-67	เพิ่มส่วนลดแบบใหม่ (เช่น ชื้อ 2 แถม 1) ได้โดยสร้าง @Component ใหม่ที่ implement interface โดยไม่แก้ OrderServiceImpl เพราะ Service รวบรวม Strategy ทั้งหมดเป็น Map ตาม DiscountType ตอนเริ่มระบบ
L Liskov Substitution	PercentDiscount, FixedAmountDiscount, NoDiscount; PendingOrderState, PaidOrderState, CancelledOrderState	OrderServiceImpl L204; OrderState L1-30	เรียก discountStrategies.get(type).calculate(...) ได้โดยไม่ต้องรู้ชนิดจริง ทุก State ทำตามสัญญาเดียวกัน คือสำเร็จหรือโยน ConflictException ไม่มีเมธอดที่โยน UnsupportedOperationException

หลักการ	ไฟล์ / คลาส	บรรทัด	เหตุผลและผลลัพธ์
I Interface Segregation	OrderState (3 เมธอด), DiscountStrategy (2 เมธอด), Service interface แยกตามโมดูล, OrderRepository	OrderState L1-30; OrderRepository L12	Interface เล็กและเฉพาะหน้าที่ ผู้ใช้ไม่ต้องพึ่งเมธอดที่ไม่ใช้ Repository ขยายเฉพาะ JpaRepository และ JpaSpecificationExecutor
D Dependency Inversion	OrderController, OrderServiceImpl, PaymentServiceImpl	OrderController L33; OrderServiceImpl L37-44; PaymentServiceImpl L33-38	Controller พึ่ง Interface OrderService Service พึ่ง DiscountStrategy และ Repository ที่เป็น Abstraction ทุกคลาสใช้ Constructor Injection
D Dependency Inversion	OrderController, OrderServiceImpl, PaymentServiceImpl	OrderController L33; OrderServiceImpl L37-44; PaymentServiceImpl L33-38	Controller พึ่ง Interface OrderService Service พึ่ง DiscountStrategy และ Repository ที่เป็น Abstraction ทุกคลาสใช้ Constructor Injection

4.2 รายงานการประยุกต์ใช้ Design Patterns

4.2.1 Enterprise / Architectural Patterns

ตารางที่ 4.2 Enterprise Patterns ที่ใช้

Pattern	การใช้งานในโครงการ
Layered Architecture	แยก controller, service, repository, domain, dto, mapper, config, exception

Pattern	การใช้งานในโครงการ
Repository	Spring Data JPA Repository ต่อ Entity และ ReportRepository สำหรับ Query รายงาน
DTO + Mapper	Request/Response Record แยกจาก Entity ผ่าน *Mapper ทุกโมดูล
Dependency Injection	Constructor Injection ทุก Service และ Controller
Specification	OrderSpecifications, ProductSpecifications, UserSpecifications ประกอบเงื่อนไขค้นหาแบบ Dynamic
Snapshot	เก็บ unit_price, ราคา Add-on และ discount_type/discount_value ไว้ในบิล

4.2.2 GoF Design Patterns (Behavioral)

ตารางที่ 4.3 GoF Patterns ที่ใช้จริงในโค้ด

Pattern	ปัญหาที่แก้	คลาสที่ใช้
Strategy	การคำนวณส่วนลดมีหลายรูปแบบ (ไม่มี, เปอร์เซ็นต์, จำนวนเงิน) ไม่ต้องการ if/else ใน Service	DiscountStrategy, NoDiscount, PercentDiscount, FixedAmountDiscount, OrderServiceImpl (เลือกจาก Map ตาม DiscountType)
State	สิ่งที่ออเดอร์ทำได้ขึ้นกับสถานะ (PENDING แก่/ชำระ/ยกเลิกได้ PAID และ CANCELLED ทำไม่ได้)	OrderState, PendingOrderState, PaidOrderState, CancelledOrderState, Order.ensureModifiable/cancel/markPaid
Observer	เมื่อชำระเงินสำเร็จต้องมีการแจ้งเตือน (บันทึก Audit Log) โดยไม่ผูกกับ PaymentService	OrderPaidEvent, ApplicationEventPublisher ใน PaymentServiceImpl, SaleAuditLogListener (@TransactionalEventListener)

Pattern	ปัญหาที่แก้	คลาสที่ใช้
Singleton	State แต่ละชนิดไม่มีข้อมูลเฉพาะตัว ใช้ซ้ำได้	PaidOrderState.INSTANCE ฯลฯ และ Spring Bean (@Service, @Component)

ตัวอย่างการทำงานของ State: เมื่อเรียก `order.markPaid()` ระบบจะส่งต่อให้ `OrderState.of(status).pay()` ถ้าสถานะเป็น PENDING จะคืน PAID ถ้าเป็น PAID หรือ CANCELLED จะโยน `ConflictException` (HTTP 409) การเพิ่มสถานะใหม่ เช่น REFUNDED ทำได้โดยเพิ่มคลาสใหม่ ไม่ต้องเพิ่ม if/else ใน Service

บทที่ 5

REST API และการจัดการข้อผิดพลาด

5.1 รายละเอียด RESTful API Endpoints

ทุก Endpoint อยู่ใต้ /api/v1 ใช้ JSON คั่น HTTP Status ตามความหมาย (200, 201, 204, 400, 401, 403, 404, 409) รายการที่คืนข้อมูลเป็นชุด (สินค้า ออเดอร์) รองรับ Pagination และ Sorting ผ่าน page, size, sort ค่าเริ่มต้นสินค้าคือ size=20 เรียงตาม name และออเดอร์คือ size=20 เรียงตาม createdAt จากใหม่ไปเก่า เอกสาร API เต็มดูได้ที่ Swagger UI

ตารางที่ 5.1 Auth และ User

Method	Endpoint	คำอธิบาย	สิทธิ์
POST	/auth/login	เข้าสู่ระบบ ออก JWT	ทุกคน
GET	/users/me	ดูโปรไฟล์ตนเอง	ล็อกอินแล้ว
PUT	/users/me/profile	แก้ไขโปรไฟล์	ล็อกอินแล้ว
PUT	/users/me/password	เปลี่ยนรหัสผ่านตนเอง	ล็อกอินแล้ว
GET	/users, /users/{id}	รายชื่อ/รายละเอียดผู้ใช้	Admin
POST	/users	เพิ่มผู้ใช้	Admin
PUT	/users/{id}	แก้ไขผู้ใช้ (Role, ข้อมูลโปรไฟล์)	Admin
PATCH	/users/{id}/status	ระงับ/เปิดใช้งานบัญชี	Admin
PATCH	/users/{id}/password	รีเซ็ตรหัสผ่านให้ผู้ใช้	Admin

ตารางที่ 5.2 ข้อมูลหลัก (Product, Category, Add-on, Promotion)

Method	Endpoint	คำอธิบาย	สิทธิ์
GET	/products (?categoryId, search, active)	รายการสินค้า พร้อมกรองและแบ่งหน้า	ล็อกอินแล้ว
GET	/products/{id}	รายละเอียดสินค้า	ล็อกอินแล้ว
POST, PUT	/products, /products/{id}	เพิ่ม/แก้ไขสินค้า (รวม addOnIds)	Admin
PATCH	/products/{id}/status	เปิด/ปิดขาย	Admin
DELETE	/products/{id}	ลบสินค้า	Admin
CRUD	/categories, /categories/{id}	จัดการหมวดหมู่	GET ล็อกอิน, อื่นๆ Admin
CRUD	/add-ons, /add-ons/{id}/status]	จัดการ Add-on	GET ล็อกอิน, อื่นๆ Admin
CRUD	/promotions, /promotions/{id}/status]	จัดการโปรโมชั่น	GET ล็อกอิน, อื่นๆ Admin

ตารางที่ 5.3 Order, Payment และ Report

Method	Endpoint	คำอธิบาย	สิทธิ์
POST	/orders	สร้างออเดอร์ (PENDING) พร้อมรายการและ Add-on	ล็อกอินแล้ว
GET	/orders (?status, from, to, cashierId)	ค้นหาบิลย้อนหลัง แบ่งหน้า	ล็อกอินแล้ว
GET	/orders/{id}	รายละเอียดออเดอร์	ล็อกอินแล้ว
PUT	/orders/{id}/items	แก้ไข/ลบรายการ (เฉพาะ PENDING)	ล็อกอินแล้ว
PUT	/orders/{id}/discount	ใช้ส่วนลด (NONE, PERCENT, FIXED_AMOUNT)	ล็อกอินแล้ว

Method	Endpoint	คำอธิบาย	สิทธิ์
POST	/orders/{id}/cancel	ยกเลิกออเดอร์ (เฉพาะ PENDING)	ล็อกอินแล้ว
GET	/orders/stats/today	สถิติยอดขายวันนี้	ล็อกอินแล้ว
POST	/orders/{orderId}/payment	ชำระเงิน (CASH, QR_CODE, CARD)	ล็อกอินแล้ว
GET	/orders/{orderId}/payment	ดูข้อมูลการชำระเงินใบเสร็จ	ล็อกอินแล้ว
GET	/reports/sales-summary, top-products, sales-by-cashier, sales-by-payment-method, sales-by-day, sales-by-category	รายงานสรุбыอดขาย 6 รูปแบบ	Admin

ตัวอย่างคำขอชำระเงินด้วยเงินสด:

```
POST /api/v1/orders/42/payment

Authorization: Bearer <token>

{ "method": "CASH", "amountReceived": 200.00 }
```

5.2 การตรวจสอบข้อมูลและการจัดการ Exception

ข้อมูลขาเข้าตรวจด้วย Bean Validation ที่ Request DTO และตรวจกฎธุรกิจใน Service (เช่น ส่วนลดเปอร์เซ็นต์ต้องไม่เกิน 100 จำนวนเงินต้องไม่เกินยอดรวม เงินที่รับต้องไม่น้อยกว่ายอดสุทธิ) ข้อผิดพลาดทั้งหมดถูกจัดการที่ GlobalExceptionHandler (@RestControllerAdvice) และตอบกลับด้วยรูปแบบเดียวกันทุกกรณี

```
{
  "timestamp": "2026-10-09T08:24:00Z",
  "status": 409,
  "error": "Conflict",
  "message": "Order is already paid and cannot be changed",
}
```

```

"path": "/api/v1/orders/42/items",

"fieldErrors": {}

}

```

ตารางที่ 5.4 การแปลง Exception เป็น HTTP Status

Exception	HTTP Status	ตัวอย่างสถานการณ์
ResourceNotFoundException	404 Not Found	ไม่พบออเดอร์หรือสินค้าตาม id
BadRequestException	400 Bad Request	เงินที่รับน้อยกว่ายอดสุทธิ, ส่วนลดเกินยอดรวม
ConflictException	409 Conflict	แก้ไข/ชำระ/ยกเลิกออเดอร์ที่ไม่ใช่ PENDING, ชื่อหมวดหมู่ซ้ำ
DataIntegrityViolationException	409 Conflict	ชำระเงินซ้ำพร้อมกัน (payments.order_id เป็น UNIQUE)
MethodArgumentNotValidException	400 Bad Request	ข้อมูลไม่ผ่าน Bean Validation (มี fieldErrors)
HttpMessageNotReadableException, MethodArgumentTypeMismatchException	400 Bad Request	JSON หรือชนิดพารามิเตอร์ไม่ถูกต้อง
BadCredentialsException	401 Unauthorized	Username หรือรหัสผ่านไม่ถูกต้อง
ไม่มี Token / สิทธิ์ไม่พอ (Security)	401 / 403	เรียก API โดยไม่ล็อกอิน / Cashier เรียก API ของ Admin
Exception อื่นๆ	500 Internal Server Error	ข้อผิดพลาดที่ไม่คาดคิด (ไม่เปิดเผยรายละเอียด)

5.3 Database Migration

โครงสร้างฐานข้อมูลและข้อมูลตั้งต้นจัดการด้วย Flyway ไฟล์อยู่ที่
code/backend/src/main/resources/db/migration ระบบรัน Migration อัตโนมัติเมื่อเริ่มต้น

ตารางที่ 5.5 Flyway Migration

ไฟล์	หน้าที่
V1__init_schema.sql	สร้างตารางหลัก พร้อม Constraint, CHECK และ Index
V2__seed_demo_data.sql	ข้อมูลตัวอย่าง (ผู้ใช้ หมวดหมู่ สินค้า Add-on)
V3__order_item_add_ons.sql	ตาราง order_item_add_ons เก็บ Add-on ที่เลือกพร้อมราคา Snapshot
V4__order_discount_rule.sql	เพิ่ม discount_type, discount_value ใน orders เพื่อคำนวณส่วนลดใหม่เมื่อแก้รายการ
V5__promotions.sql	ตาราง promotions

บทที่ 6

กระบวนการทำงาน การทดสอบ และการ Deploy

6.1 Git Workflow และ Branching Strategy

- main: เวอร์ชันสำหรับส่งมอบและใช้งานจริง รวมโค้ดผ่าน Pull Request เท่านั้น
- develop: Branch สำหรับรวมงานของสมาชิกทุกคน
- Branch ส่วนตัวตั้งชื่อ ชื่อ_รหัสนักศึกษา_section แล้วเปิด Pull Request เข้า develop โดยมี Reviewer อย่างน้อย 1 คน
- รูปแบบ Commit: <type>: <สิ่งที่ทำ> โดย type ได้แก่ feat, fix, refactor, test, docs, chore

ตารางที่ 6.1 Branch ของสมาชิก

สมาชิก	Branch
ภาควัฒน์ สุขมณี	Phakawat_6733804189_04
ธนันต์ สวีกัน	Thana-nan_6733805868_04
กวินธิดา อนุวิวัฒน์	Kawinthida_6733803905_04
กัญญาวีร์ สิงห์ลี	Kanyawee_6733805737_04

6.2 การทดสอบระบบ

โค้ดทดสอบอยู่ที่ code/backend/src/test/java มี 21 คลาส ประมาณ 119 กรณีทดสอบ แบ่งเป็น 3 ระดับ

ตารางที่ 6.2 ชุดการทดสอบ

ระดับ	คลาสตัวอย่าง	สิ่งที่ทดสอบ
Unit Test (Mockito)	OrderServiceImplTest, ProductServiceImplTest, CategoryServiceImplTest, AddOnServiceImplTest, UserServiceImplTest, ReportServiceImplTest	ตรรกะ Service โดย Mock Repository เช่น การคำนวณยอด การ Snapshot ราคา กฎของส่วนลด
Domain Test	DiscountStrategyTest, OrderStateTest, OrderItemAddOnMappingTest	Strategy ส่วนลด การเปลี่ยนสถานะออเดอร์ และการ Mapping Add-on
Controller / Integration Test	OrderApiIntegrationTest, PaymentApiIntegrationTest, ProductApiIntegrationTest, ReportApiIntegrationTest, SecurityIntegrationTest, *ControllerTest, CafePosApplicationTests	การเรียก API จริง ผ่าน Security และ Validation กับฐานข้อมูล H2

```
cd code/backend
```

```
./mvnw test
```

รายงานผลการทดสอบ (surefire report) เก็บไว้ที่โฟลเดอร์ test/ ของ Repository

6.3 Docker, CI และการ Deploy

6.3.1 Docker

code/Dockerfile เป็น Multi-stage Build 3 ขั้น (1) build React ด้วย Node 24 (2) คัดลอกผลลัพธ์เข้า src/main/resources/static แล้ว build Spring Boot ด้วย Maven และ Temurin 21 (3) รันด้วย eclipse-temurin:21-jre-alpine ผลลัพธ์คือ image เดียวที่ให้บริการทั้ง API และหน้าเว็บบนพอร์ต 8080 ส่วน code/docker-compose.yml ประกอบด้วยบริการ db (PostgreSQL 17 พร้อม Health Check) และ app

6.3.2 Continuous Integration

ไฟล์ .github/workflows/ci.yml ทำงานเมื่อ Push เข้า main หรือ develop และเมื่อเปิด Pull Request ประกอบด้วย 2 งาน คือ Backend (ตั้งค่า Java 21 แล้วรัน ./mvnw -B verify ซึ่งคอมไพล์และรัน Test ทั้งหมด) และ Frontend (Node 24, npm ci, npm run build)

6.3.3 การ Deploy

ระบบ Deploy เป็น Docker image เดียวบน Render และใช้ PostgreSQL บน Neon กำหนดค่าเชื่อมต่อผ่านตัวแปรสภาพแวดล้อม DB_URL, DB_USERNAME, DB_PASSWORD และ PORT

ตารางที่ 6.3 ที่อยู่ของระบบที่ Deploy

รายการ	URL
เว็บแอปพลิเคชัน	[ใส่ URL ที่ Deploy บน Render]
Swagger UI	[ใส่ URL]/swagger-ui.html
Source Code	https://github.com/Mraustin0/CafePosSystem

เอกสารอ้างอิง

- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- PlantUML. (n.d.). *PlantUML language reference guide*. <https://plantuml.com/>
- PostgreSQL. (n.d.). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>
- React. (n.d.). *React documentation*. <https://react.dev/>
- Red Gate Software. (n.d.). *Flyway documentation*. <https://documentation.red-gate.com/flyway>
- Spring. (n.d.). *Spring Boot reference documentation*. <https://docs.spring.io/spring-boot/>
- Spring. (n.d.). *Spring Security reference*. <https://docs.spring.io/spring-security/reference>